

Lecture 2: Core Python Syntax

Beginner Notes | Python for GenAI

1. Variables

A **variable** is a name used to store or refer to a value. Think of it as a labelled box.

```
name = "Rahul"  
age = 21  
print(name)  
print(age)
```

Variables can be updated. Python is case-sensitive, so **name** and **Name** are different. Use clear names such as `student_name` and `student_age`.

2. Data Types

A **data type** tells Python what kind of value we are working with.

Type	Meaning	Example
str	Text	"Hello"
int	Whole number	21
float	Decimal number	99.50
bool	True or False	True

```
name = "Rahul"  
age = 21  
price = 99.50  
is_student = True
```

```
print(type(name))  
print(type(age))
```

"21" is a string, while **21** is an integer.

3. User Input

input() lets a program receive information while it is running.

```
name = input("Enter your name: ")  
print("Hello", name)
```

Important: `input()` returns the user's input as a string by default, even when the user types a number.

4. Type Casting

Type casting means converting a value from one data type to another.

Function	Purpose	Example
<code>int()</code>	Convert to integer	<code>int("21")</code>
<code>float()</code>	Convert to decimal	<code>float("99.50")</code>
<code>str()</code>	Convert to string	<code>str(21)</code>
<code>bool()</code>	Convert to boolean	<code>bool(1)</code>

```
age = int(input("Enter your age: "))  
print("Next year you will be", age + 1)
```

5. Operators

Operators are symbols or keywords used to perform operations.

Arithmetic operators

Operator	Meaning	Example
+	Addition	$10 + 3 \rightarrow 13$

Operator	Meaning	Example
-	Subtraction	10 - 3 → 7
*	Multiplication	10 * 3 → 30
/	Division	10 / 3 → 3.333...
//	Floor division	10 // 3 → 3
%	Remainder	10 % 3 → 1
**	Power	10 ** 3 → 1000

Comparison operators return True or False.

Operator	Meaning
==	Equal to
!=	Not equal to
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to

Logical operators

Operator	Meaning
and	True when both conditions are True.
or	True when at least one condition is True.
not	Reverses True and False.

Assignment operators include =, +=, -=, *=, /=. They are useful for updating values.

6. Strings

A **string** is text made up of characters. Strings use single or double quotation marks.

```
word = "Python"
print(word[0])
print(word[-1])
print(word[0:3])
```

Python uses **zero-based indexing**, so the first character is at position 0. Negative indexing starts from the end. In a slice **start:end**, the end position is not included.

7. String Methods

String methods are built-in operations for working with text. They use dot notation, such as **text.upper()**.

Method	Meaning	Example
upper()	Convert to uppercase	"hello".upper()
lower()	Convert to lowercase	"HELLO".lower()
title()	Capitalize each word	"hello python".title()
capitalize()	Capitalize first character	"hello".capitalize()
strip()	Remove leading/trailing spaces	" hello ".strip()
replace()	Replace text	text.replace("Java", "Python")
split()	Split text into parts	"Python is easy".split()
find()	Find the position of text	text.find("Python")
startswith()	Check starting text	text.startswith("python")
endswith()	Check ending text	text.endswith(".py")
count()	Count occurrences	"banana".count("a")
join()	Join strings with a separator	"".join(["A", "B", "C"])

8. f-Strings

An **f-string** makes it easy to place variables and expressions inside text. Put **f** before the opening quote and values inside **{ }**.

```
name = "Rahul"
age = 21
print(f"My name is {name} and I am {age} years old.")

a = 10
b = 20
print(f"The sum is {a + b}")

price = 99.5678
print(f"Price: {price:.2f}")
```

9. Docstrings

A **docstring** is documentation written inside a module, function, or class. It is commonly written using triple quotes.

```
def greet():
    """Display a simple greeting."""
    print("Hello!")

greet()
print(greet.__doc__)
```

A comment is mainly a note for readers. A docstring is documentation associated with code such as a function, class, or module.

10. Common Pattern

A useful pattern is **Input** → **Convert** → **Process** → **Display**.

```
name = input("Enter your name: ")
age = int(input("Enter your age: "))
city = input("Enter your city: ")
print(f"Name: {name.title()}")
print(f"Age: {age}")
print(f"City: {city.title()}")
print(f"Next year you will be {age + 1}.")
```

11. Quick Cheat Sheet

Concept	Remember
variable	A name used to store or refer to a value.
str / int / float / bool	Common basic data types.
input()	Gets user input as a string by default.
int()/float()/str()	Convert common data types.
==, !=, >, <, >=, <=	Compare values.
and / or / not	Logical operators.
string[index]	Access a character.
string[start:end]	Take a slice.
upper(), lower(), title(), strip()	Common text-cleaning methods.
replace(), split(), find()	Common text-processing methods.
f"...{value}..."	Insert values into text.
""" ... """	Common docstring syntax.